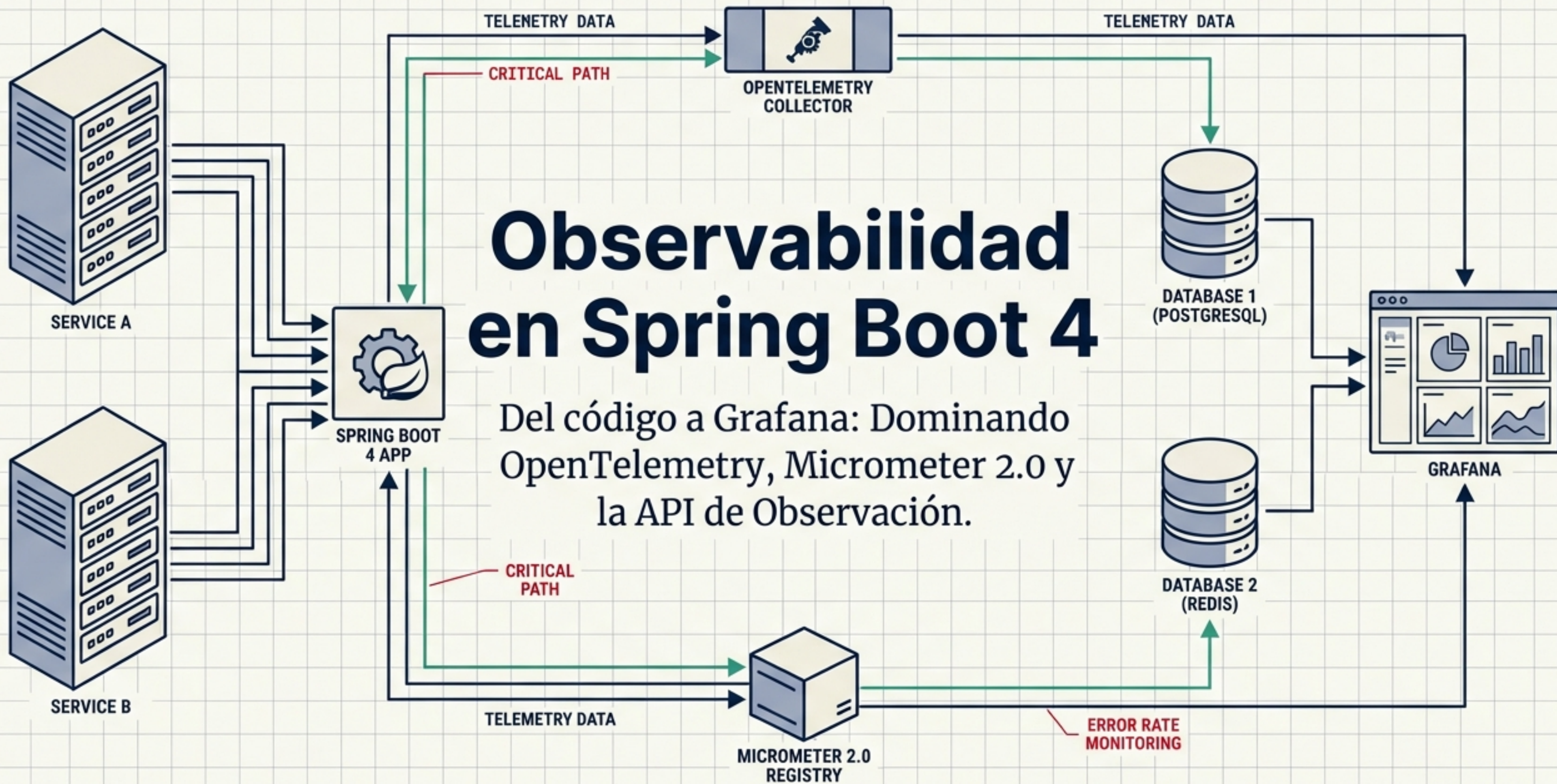


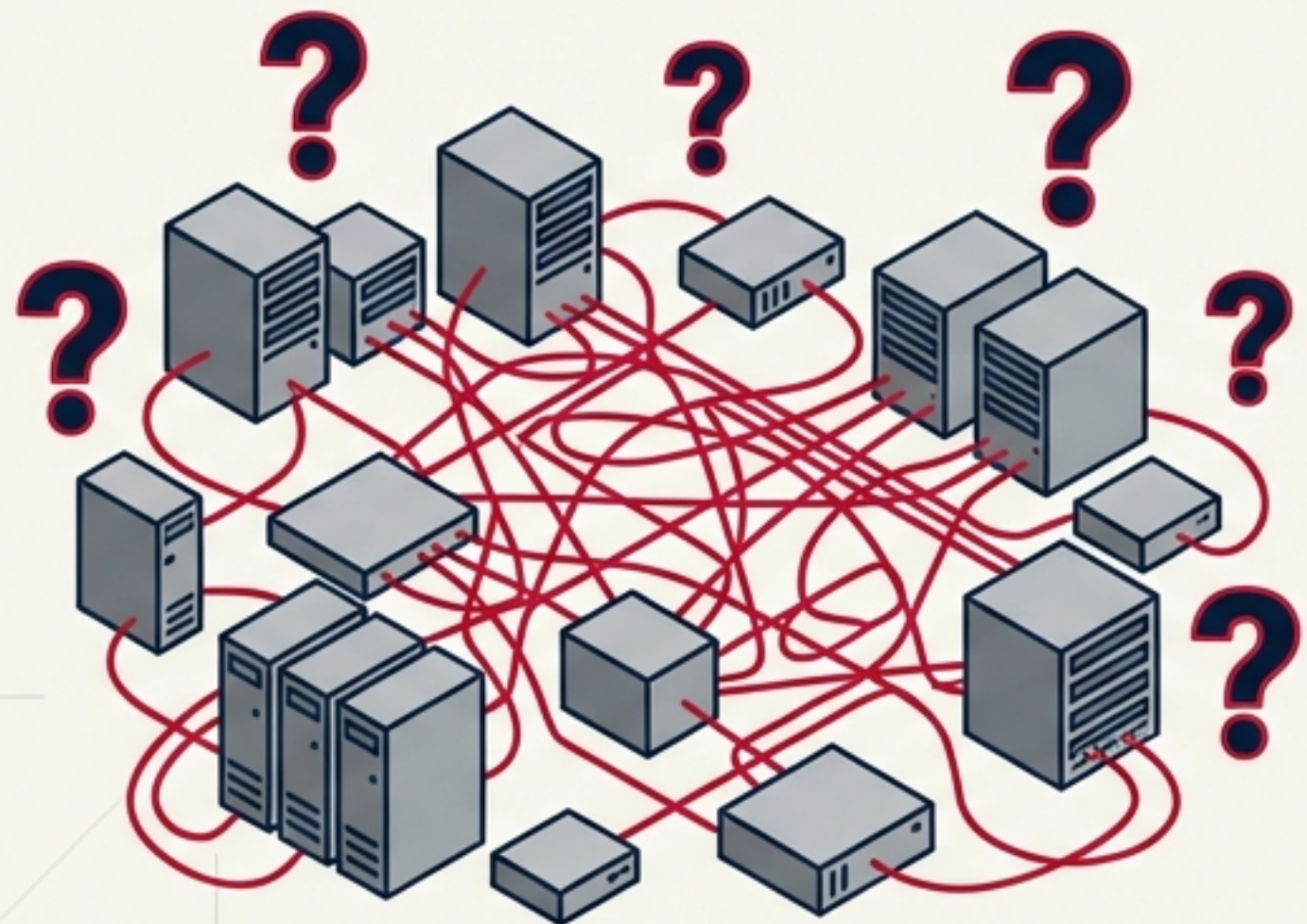
Observabilidad en Spring Boot 4

Del código a Grafana: Dominando OpenTelemetry, Micrometer 2.0 y la API de Observación.



Volar a ciegas en sistemas distribuidos

“La esperanza no es una estrategia de producción.”



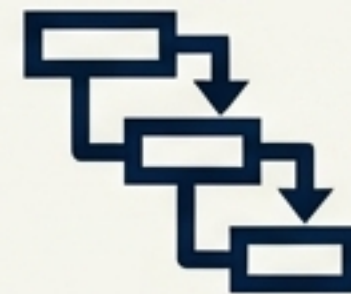
Logs

¿Qué sucedió?
Eventos discretos
con contexto
temporal.



Métricas

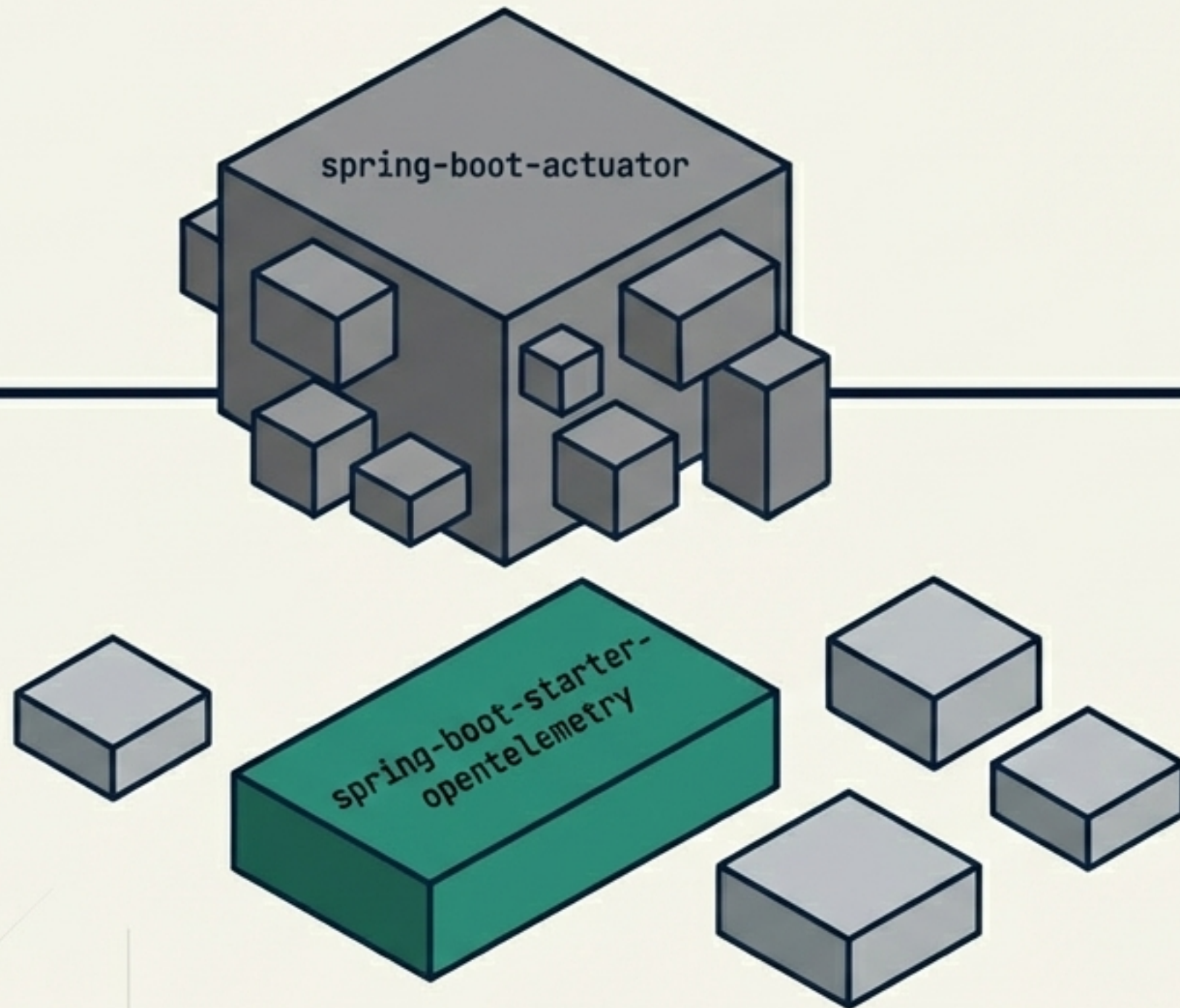
¿Cuánto y con
qué frecuencia?
Agregaciones
numéricas en el
tiempo.



Trazas

¿Dónde pasamos
el tiempo? El
flujo de la petición
a través de los
servicios.

La evolución hacia la modularidad



Antes (Spring Boot 3)

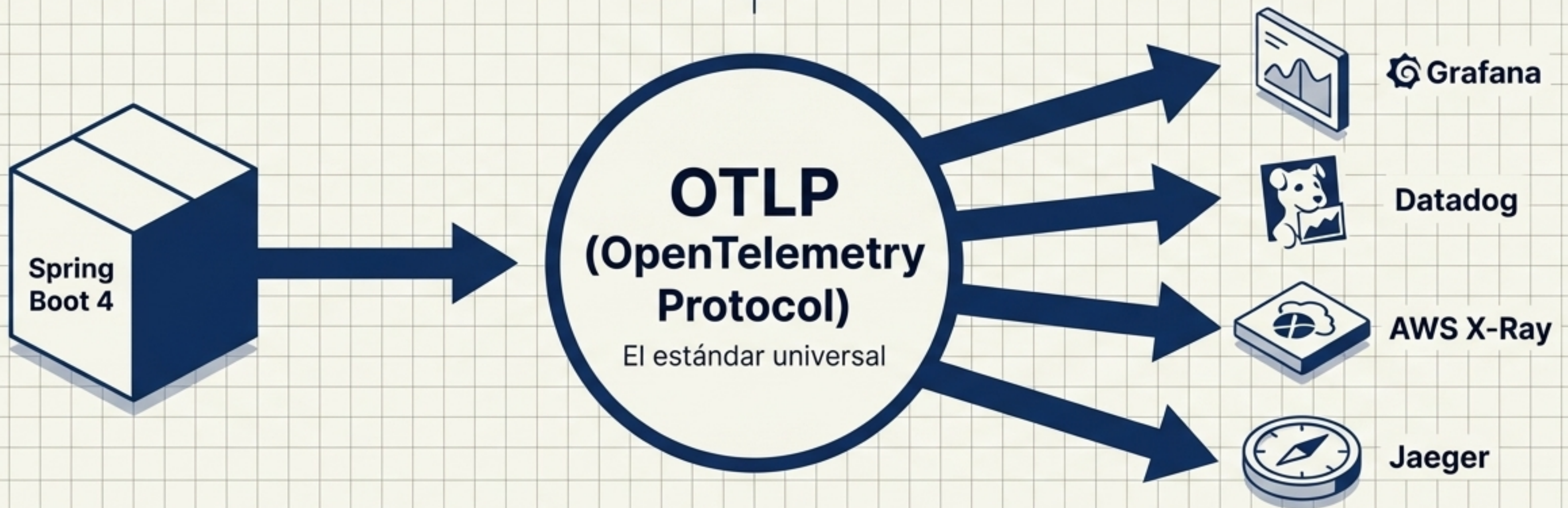
Observabilidad significaba importar todo el **módulo monolítico de Actuator**, incluyendo endpoints internos innecesarios.

Ahora (Spring Boot 4)

Arquitectura enfocada y ligera.

Separación estricta de responsabilidades. El nuevo estándar: El **módulo independiente de OpenTelemetry** permite instrumentar sin sobrecargar el classpath.

El protocolo es lo que importa, no la biblioteca



Estándar Vendor-Neutral

OpenTelemetry es el lenguaje universal para emitir telemetría.

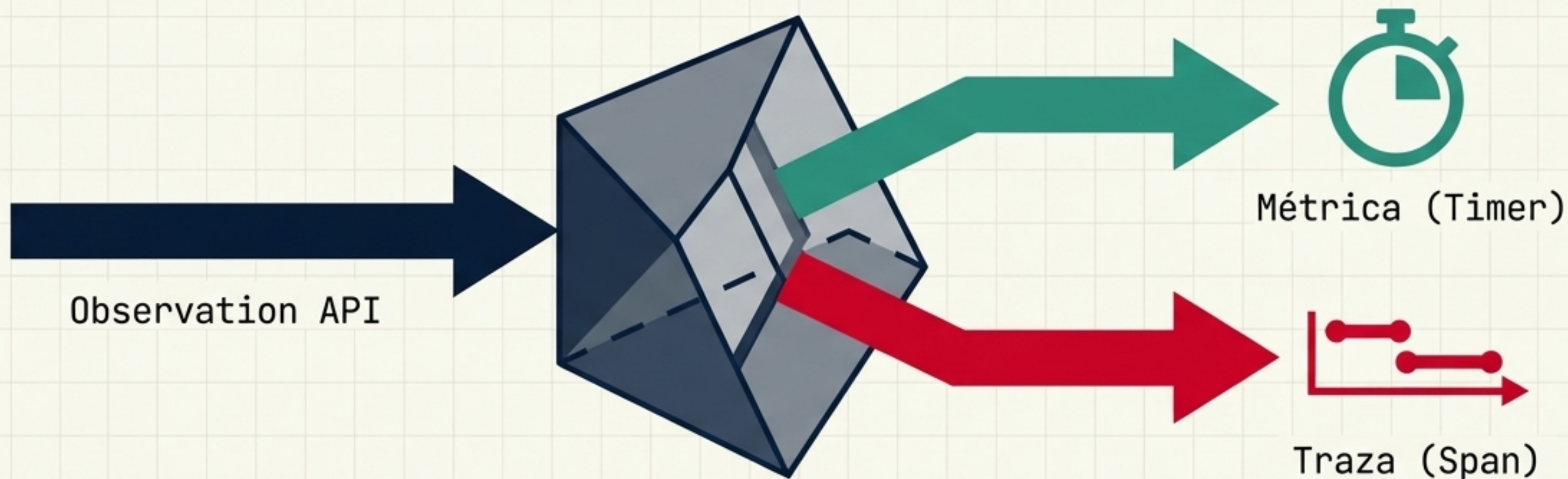
Exportación OTLP Nativa

Spring Boot 4 exporta métricas, trazas y logs mediante **OTLP nativamente**.

Libertad Total

Cambiar de backend de observabilidad ya no requiere refactorizar el código Java.

El motor unificado: Micrometer 2.0



El viejo paradigma

Instrumentar el código **dos veces**: una vez para contar tiempos, y otra vez para trazar la ruta.

Observation API

Una **única API** para gobernarlos a todos. Un solo punto de instrumentación genera simultáneamente la métrica de Prometheus y el **Span** de OpenTelemetry.

La varita mágica de la instrumentación

```
@RestController
public class HomeController {

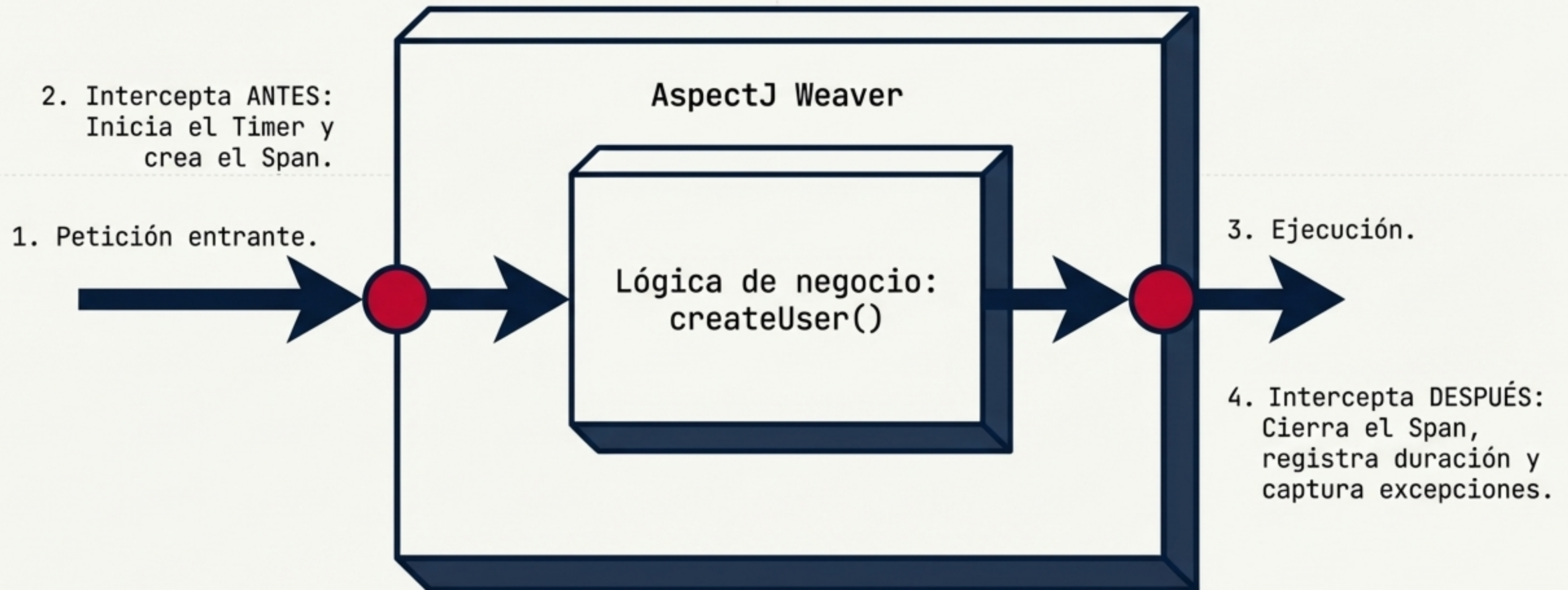
    private static final Logger log = LoggerFactory.getLogger(HomeController.class);

    @Observed(name = "user.creation")
    @PostMapping("/users")
    public String createUser(@RequestBody User user) {
        log.info("Creando nuevo usuario en el sistema");
        // Business logic...
        return "Usuario creado";
    }
}
```

**Genera automáticamente
un Timer
(user.creation.duration)
y un Span para la traza**

El eslabón perdido: Por qué necesitas AOP

Sin AspectJ, la anotación `@Observed` es solo metadatos inútiles. Spring necesita interceptar la llamada en tiempo de ejecución utilizando el paradigma de Programación Orientada a Aspectos.



La receta completa de dependencias

```
<!-- El nuevo starter nativo -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-opentelemetry</arti
</dependency>

<!-- El interceptor mágico -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aspectj</
artifactId> </dependency>
```

- ✓ Generación nativa de telemetría sin Actuator.
- ✓ Intercepción automática en controladores y métodos `@Observed`.
- ✓ Integración de contexto W3C out-of-the-box.

Exportando la telemetría: El puente OTLP

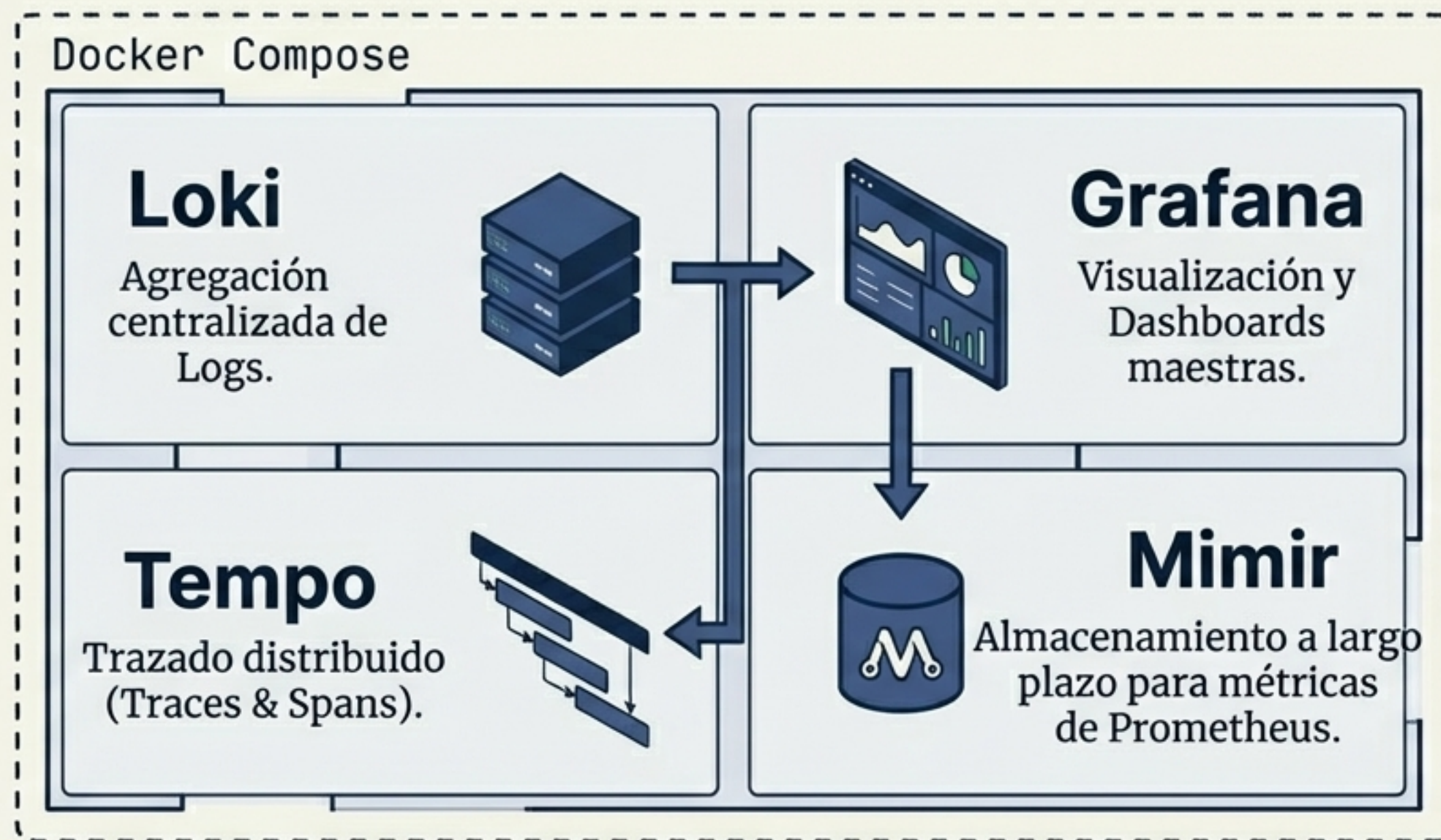
```
spring:
  application:
    name: "user-service"
  otlp:
    metrics:
      export:
        url: "http://localhost:4318/v1/metrics"
  opentelemetry:
    tracing:
      export:
        otlp:
          endpoint: "http://localhost:4318/v1/traces"

management:
  tracing:
    sampling:
      probability: 1.0 # ¡Advertencia de Producción!
```



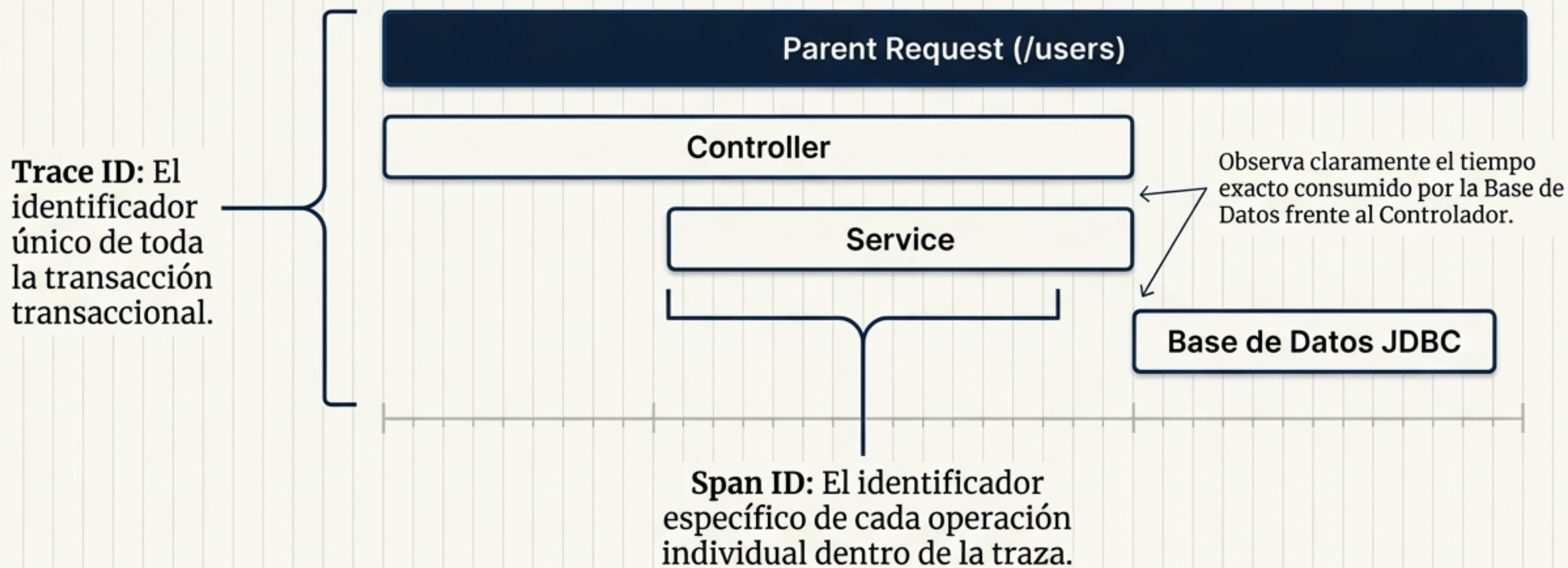
Muestreo (Sampling) al 1.0 (100%) es ideal para desarrollo, pero destruirá tu presupuesto en producción. El valor por defecto es 0.1 (10%).

El Destino: El Stack LGTM de Grafana



Con spring-boot-docker-compose, Spring detecta OpenTelemetry y levanta automáticamente este stack completo al ejecutar tu aplicación en local. ¡Cero configuración manual!

Visualización del viaje: Trazas y Spans y Spans

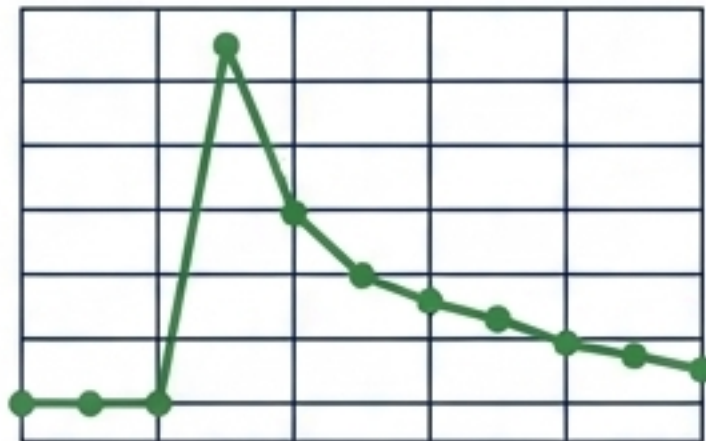


Cuantificando el comportamiento: Métricas

4.57



Time to last call to GET /expenses



Total Requests to POST /expenses

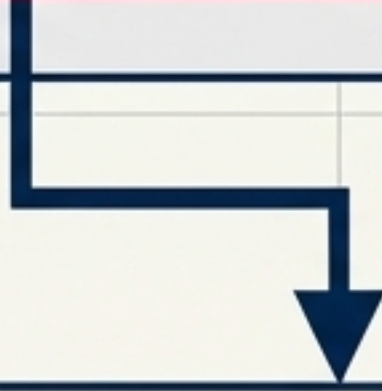


- Las métricas de latencia p99 demuestran el tiempo de respuesta del 1% de los usuarios más lentos.
- La métrica generada por `@Observed` aparece como `user.creation.duration`.
- Métricas predeterminadas de la JVM (uso de memoria, recolección de basura) vienen incluidas de forma nativa sin código extra.

Correlación de Logs: El Santo Grial del Debugging

Terminal Window

2026-10-15T11:30:05.801 INFO **[64a2b1c8f9, 8d7a12b]** dev.app.HomeController: Creando nuevo usuario...

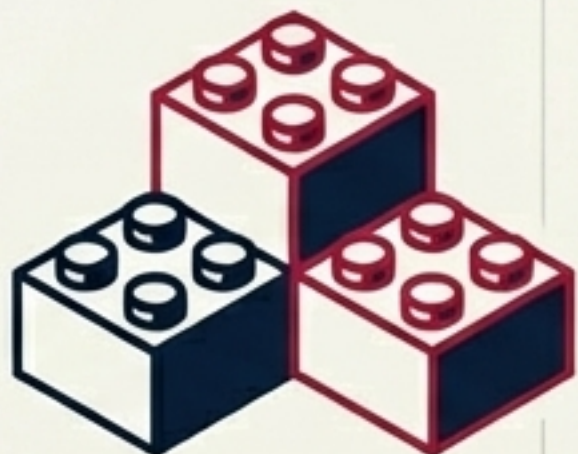


El bloque [traceId, spanId] se **inyecta** automáticamente. Si **una petición falla**, ya no buscas **ciegamente** en miles de líneas. Copias el **Trace ID** de Grafana, lo filtras en tus logs y ves la historia exacta de ese usuario en particular.

Matriz de Decisión Arquitectónica

Dimensión	starter-opentelemetry (Spring Boot 4)	starter-actuator (Clásico)
Caso de Uso Principal	Telemetría completa, trazado distribuido de alta fidelidad	Endpoints de gestión, Health checks, Info interna
Estilo de Exportación	Push activo (Protocolo OTLP)	Pull pasivo (Scraping de Prometheus)
Filosofía	Vendor-neutral, estándar de la industria OTel	Acoplado al ecosistema Spring clásico
Nivel de intrusión	Ligero, modular, ideal para GraalVM	Pesado, expone endpoints HTTP adicionales

Reglas de Oro para Producción



1. Manténlo modular

Usa el starter de OTel a menos que estrictamente necesites los endpoints de Actuator.



2. Nunca olvides AOP

`@Observed` sin dependencias de AspectJ es código muerto.



3. Muestrea inteligentemente

Configura `sampling.probability` en función de tu tráfico real, no traces el 100% de tus peticiones.

Construir es solo el 10%. Operar lo es todo.